Computer Science & Engineering

Operating Systems – Assignment No. 3 – Case Study

Problem 5: Banking and ATM Transaction Processing System

Linked mini-project: OS Algorithm Visualizer (Practical No. 9)

5th Sem, Section B · Subject Teacher: Dr. Pallavi Wankhede · Academic Year 2026–27

Submitted by :

1. ROHINI SHENDE (Roll No 24003173)

2. VIRAJ MOHADIKAR (Roll No 24003158)

1. Selected Problem Statement and Real-World Context

Problem 5 – Banking and ATM Transaction Processing System. A bank processes ATM withdrawals, deposits, fund transfers, balance inquiries and transaction records at the same time. Many transactions execute concurrently and need database access, transaction-processing services and memory. Incoming requests must be queued and processed without data inconsistency. Banker's Algorithm helps ensure that shared resources are allocated safely.

Challenge: schedule transactions, synchronize request processing, manage memory allocation, and use Banker's Algorithm to evaluate safe allocation of limited resources.

2. Problem Analysis, Objectives, Constraints and Assumptions

Main problems: concurrent transactions compete for CPU and resources; urgent transfers may wait behind longer jobs; requests can be lost or duplicated without synchronization; poor memory placement can leave a transaction waiting; and unsafe resource allocation can lead to deadlock.

Objectives: serve urgent transactions first, preserve request order, allocate memory with minimum waste, and grant shared resources only when the system remains safe.

Constraints: five transactions, three shared resource types, five memory partitions totaling 1700 KB, a bounded request buffer and one CPU. **Assumptions:** priority 1 is highest; maximum resource claims are declared in advance; memory uses fixed partitions; scheduling is non-preemptive unless stated.

OS Resource Management for a Banking & ATM Transaction System

Selected problem: Problem 5 — Banking and ATM Transaction Processing System. ATMs, mobile apps, online banking and branch counters generate concurrent transactions. These requests must be scheduled, synchronized, protected from deadlock and provided with memory. This report applies CPU Scheduling, Producer–Consumer synchronization, Banker's Algorithm and Memory Management.

1. Problem Analysis

Objectives: minimize waiting and turnaround time while allowing urgent transactions to run first; pass each request from channel to processor exactly once; avoid unsafe resource allocation; and allocate session memory with minimum waste.

Constraints: five transactions, 10 database connections, 5 transaction-service threads, 7 ledger-lock/printer units, a bounded request buffer, and memory partitions of 100, 500, 200, 300 and 600 KB.

Assumptions

- CPU burst time, priority and maximum resource claim are known at admission time.
- Priority 1 is the highest priority; fund transfers are urgent.
- Mutex and counting-semaphore primitives are available.
- Each transaction does not request more than its declared maximum resource claim.

2. Transaction Details

Txn	Transaction	AT	BT	Priority	Memory (KB)	Max claim (A,B,C)	Allocation
T1	ATM Withdrawal	0	5	2	212	(7,5,3)	(0,1,0)
T2	Fund Transfer	1	3	1	417	(3,2,2)	(2,0,0)
T3	Cash Deposit	2	8	3	112	(9,0,2)	(3,0,2)
T4	Statement / Record	3	6	5	426	(2,2,2)	(2,1,1)
T5	Balance Inquiry	4	2	4	90	(4,3,3)	(0,0,2)

Resources: A = DB connections (10), B = transaction-service threads (5), C = ledger locks (7). Available = (3,3,2). Memory partitions: 100, 500, 200, 300 and 600 KB (total 1700 KB).

3. CPU Scheduling — Priority (Non-preemptive)

When the CPU becomes free, the ready transaction with the smallest priority number runs. T1 starts first because it is the only transaction present at time 0. Once T1 completes, the highest-priority waiting transaction T2 runs, followed by T3, T5 and T4.

Gantt chart: T1 (0–5) → T2 (5–8) → T3 (8–16) → T5 (16–18) → T4 (18–24).

Waiting times: T1 = 0, T2 = 4, T3 = 6, T4 = 15, T5 = 12. Average waiting time = (0+4+6+15+12)/5 = **7.4 time units**.

4. Process Synchronization — Producer–Consumer

ATM, mobile and online channels act as producers, while transaction workers act as consumers. A bounded buffer temporarily stores requests. Semaphores prevent producers from inserting into a full buffer and consumers from removing from an empty buffer. A mutex protects the shared buffer from race conditions.

Initialization: empty = N, full = 0, mutex = 1.

Producer: wait(empty) → wait(mutex) → insert request → signal(mutex) → signal(full).

Consumer: wait(full) → wait(mutex) → remove request → signal(mutex) → signal(empty) → process request.

Worked trace (buffer size N = 2)

Tick	Thread	Operation	Empty	Full	Buffer
1	C1	Consumer blocks because buffer is empty	2	0	[]
2	P1	Insert T1	1	1	[T1]
3	P1	Insert T2	0	2	[T1, T2]
4–5	P1	Producer blocks because buffer is full	0	2	[T1, T2]
6	C1	Remove T1	1	1	[T2]
7	C1	Remove T2	2	0	[]
8–9	P1	Insert T3 and T4	0	2	[T3, T4]
10–11	C1	Remove T3 and T4	2	0	[]
12–13	P1/C1	Insert and remove T5	2	0	[]

The buffer follows FIFO order. The mutex prevents simultaneous access, and the empty/full semaphores prevent overflow and underflow. The tested buffer maintains $0 \le \text{items} \le N$ and no request is lost or duplicated.

5. Deadlock Avoidance — Banker's Algorithm

Banker's Algorithm grants a resource request only if the resulting system remains in a safe state. Need = Maximum − Allocation. A request larger than Need is an error; a request larger than Available must wait; otherwise, the allocation is tested and granted only if the state stays safe.

Need matrix

Process	Allocation (A,B,C)	Maximum (A,B,C)	Need (A,B,C)
T1	(0,1,0)	(7,5,3)	(7,4,3)
T2	(2,0,0)	(3,2,2)	(1,2,2)
T3	(3,0,2)	(9,0,2)	(6,0,0)
T4	(2,1,1)	(2,2,2)	(0,1,1)
T5	(0,0,2)	(4,3,3)	(4,3,1)

Available = (3,3,2). Safe sequence: T2 → T4 → T1 → T3 → T5. Since all transactions can finish in this order, the initial state is safe.

Banker's Safety Algorithm — Worked Trace

Step	Work before	Process	Need ≤ Work?	Work after
1	(3,3,2)	T2	(1,2,2) ≤ (3,3,2)	(5,3,2)
2	(5,3,2)	T4	(0,1,1) ≤ (5,3,2)	(7,4,3)
3	(7,4,3)	T1	(7,4,3) ≤ (7,4,3)	(7,5,3)
4	(7,5,3)	T3	(6,0,0) ≤ (7,5,3)	(10,5,5)
5	(10,5,5)	T5	(4,3,1) ≤ (10,5,5)	(10,5,7)

Example runtime requests

Process	Request	Available before	Decision	Reason
T2	(1,0,2)	(3,3,2)	Granted	Request is within Need and Available; new state is safe.
T5	(3,3,0)	(2,3,0)	Wait	Needs more resource A than is currently available.
T1	(0,2,0)	(2,3,0)	Denied (unsafe)	After tentative allocation, no process can satisfy its Need.

6. Memory Management — First Fit, Best Fit and Worst Fit

The free memory partitions are 100, 500, 200, 300 and 600 KB. Each transaction is placed in a partition that can hold its memory request. First Fit selects the first suitable partition, Best Fit selects the smallest suitable partition, and Worst Fit selects the largest suitable partition.

Strategy	Placement	Internal fragmentation	Free partition	Result
First Fit	T1→500, T2→600, T3→200, T5→100	569 KB	300 KB	T4 (426 KB) waits
Best Fit	T1→300, T2→500, T3→200, T4→600, T5→100	443 KB	0 KB	All five transactions placed
Worst Fit	T1→600, T2→500, T3→300, T5→200	769 KB	100 KB	T4 (426 KB) waits

Best Fit serves all five transactions with lower internal fragmentation in this example. First Fit and Worst Fit leave T4 waiting because no remaining single partition is large enough for its 426 KB request. This is poor packing; total free memory alone does not guarantee that one suitable partition is available.

7. Test Cases and Results

Test	Module / Input	Expected result
TC1	Priority scheduling: five transactions	Average WT = 7.4; average TAT = 12.2 time units.
TC2	Banker's safety check	Safe sequence: T2 → T4 → T1 → T3 → T5.
TC3	Banker's runtime requests	T2 request granted; T5 waits; T1 request denied as unsafe.
TC4	Memory: requests 212, 417, 112, 426, 90 KB	Best Fit places all five; First Fit and Worst Fit leave T4 waiting.
TC5	Producer–Consumer bounded buffer	FIFO order preserved; no overflow, underflow, lost or duplicated request.

8. Algorithm Design and System Integration

The transaction processing pipeline combines all four OS mechanisms:

Step	Action
1. Admission	ATM, mobile, online and branch channels produce requests into the bounded buffer.
2. Synchronization	Producer–Consumer semaphores and mutex preserve safe FIFO access to the queue.
3. Memory allocation	The selected fit strategy allocates a memory partition to each transaction.
4. Resource safety	Banker's Algorithm checks requests and grants resources only if the state remains safe.
5. CPU scheduling	Eligible transactions are scheduled by priority; smaller priority number means higher priority.
6. Completion	After processing, the transaction releases its memory and shared resources.

Integration comparison

Integrated run	Transactions completed	Average WT	Average TAT	Waiting
Best Fit + Priority	T1, T2, T3, T4, T5	7.4	12.2	None
First Fit + Priority	T1, T2, T3, T5	5.5	10.0	T4 (no memory)

The First Fit averages look lower only because T4 is excluded from the calculation. Best Fit is preferable here because it allows every transaction to be placed and completed. This demonstrates that memory allocation, synchronization, deadlock avoidance and CPU scheduling must be considered together.

9. Conclusion

This case study applies four Operating System concepts to a banking and ATM transaction system. Priority scheduling serves urgent fund transfers first, Producer–Consumer synchronization protects the shared request queue, Banker's Algorithm checks safe resource allocation, and Best Fit memory management places all five transactions in the given example. The safe sequence is T2 → T4 → T1 → T3 → T5, and Best Fit has 443 KB internal fragmentation compared with 569 KB for First Fit and 769 KB for Worst Fit. Overall, the study shows that OS algorithms work together to improve safe and efficient transaction processing.

Submitted by: Rohini Shende and Viraj Mohadikar